

Lab Report: Parameterized Architecture and Propagation Delay Analysis in Ripple-Carry Adders

Digital Design Verification Lab

May 14, 2026

Abstract

This report analyzes the structural generation of a parameterized N -bit Ripple-Carry Adder (RCA) using SystemVerilog. It documents the methodology for modeling physical inertial gate delays within the Icarus Verilog simulation environment and examines the resulting worst-case propagation delay. Furthermore, it explores the manifestation of uninitialized unknown ('X') states during power-on simulation and their implications for hardware reset strategies.

1 Parameterized Structural Modeling

Hardcoding bus widths limits the reusability of digital IP. SystemVerilog resolves this through the use of compile-time parameters.

- **parameter:** Declared within a module to define flexible constants (e.g., `xWIDTH = 4`). When instantiating the module, the parameter can be overridden using the `#(.PARAM_NAME(value))` syntax.
- **genvar and Generate Loops:** Hardware cannot be instantiated dynamically at runtime. The `genvar` variable combined with a `for` loop instructs the synthesis tool to literally stamp out physical copies of a sub-module (like a 1-bit full adder) at compile time, wiring them together automatically based on the loop index.

2 Modeling Physical Gate Delays

While SystemVerilog allows zero-time modeling via delta-cycles, analyzing timing bottlenecks requires artificial gate delays.

- **The 'timescale Directive:** Declared as `'timescale 1ns/1ps`, this maps integer delays (e.g., `#1`) to actual physical time units (1 nanosecond), with a calculation precision of 1 picosecond.
- **The wire vs. logic Quirk:** The Icarus Verilog execution engine handles the modern `logic` data type primarily as a procedural variable. To force Icarus to respect continuous assignment delays (`assign #1`), the driven signals *must* be explicitly declared as continuous `wire` nets.

3 Propagation Analysis and Worst-Case Delay

The defining characteristic of an RCA is its linear $\mathcal{O}(N)$ timing performance. The N th output bit cannot resolve until the carry bit has successfully rippled through all $N - 1$ preceding stages.

3.1 The Worst-Case Scenario

To visually prove this bottleneck in a waveform viewer, the testbench must enforce a cascade effect. By adding 1 (4'b0001) to 15 (4'b1111), every single full adder block generates a carry-out of 1, forcing the signal to propagate sequentially across the entire carry chain. In a waveform viewer, this manifests as a physical "staircase" effect, with each subsequent carry wire settling 1 nanosecond after the previous one.

4 Initialization and 'X' States

At simulation timestamp $t = 0$, internal combinatorial wires enter a "power-on limbo."

- **The Unknown State:** Even though testbench inputs are instantly valid at $t = 0$, the modeled physical gate delays (e.g., $1ns$) prevent the internal wires from resolving immediately. The simulator flags this unknown voltage level as an 'X' state (rendered as red in GTKWave).
- **Hardware Implications:** In physical silicon, transistors power up in floating, unpredictable states. The transient 'X' states observed in simulation visually demonstrate why sequential processing cores demand dedicated, robust reset signals (e.g., `rst.n`) to flush out invalid logic before clock cycles begin.

5 Source Code Implementation

```
1  'timescale 1ns/1ps
2
3  module fullAdder1bit (
4      input logic a, b, cin,
5      output wire y, cout
6  );
7      assign #1 y = (a ^ b) ^ cin;
8      assign #1 cout = (a & b) | (b & cin) | (cin & a);
9  endmodule
10
11 module nrca #(
12     parameter xWIDTH = 4
13 ) (
14     input logic [xWIDTH-1:0] A, B,
15     input logic Cin,
16     output wire [xWIDTH-1:0] Y,
17     output wire Cout
18 );
19
20     // The Internal Carry Chain
21     wire [xWIDTH:0] carry;
22     assign carry[0] = Cin;
23
24     // The Compile-Time Hardware Generator
25     genvar i;
26     for (i = 0; i < xWIDTH; i=i+1) begin
27         fullAdder1bit fa (
28             .a(A[i]),
29             .b(B[i]),
30             .cin(carry[i]),
31             .y(Y[i]),
```

```

32         .cout(carry[i+1])
33     );
34 end
35
36     assign Cout = carry[xWIDTH];
37 endmodule

```

Listing 1: SystemVerilog source for Parameterized RCA

```

1  `timescale 1ns/1ps
2
3  module tb_nrca;
4      parameter tWIDTH = 4;
5
6      logic [tWIDTH-1:0] AA, BB;
7      logic CCin;
8      logic [tWIDTH-1:0] YY;
9      logic CCout;
10
11     nrca #(.xWIDTH(tWIDTH)) dut (
12         .A(AA),
13         .B(BB),
14         .Cin(CCin),
15         .Y(YY),
16         .Cout(CCout)
17     );
18
19     initial begin
20         $dumpfile("dump.vcd");
21         $dumpvars(0, tb_nrca);
22
23         $display("Simulation Starting...");
24
25         // TESTCASE 1: Baseline zero
26         AA = 4'b0000; BB = 4'b0000; CCin = 0;
27         #10;
28
29         // TESTCASE 2: The Worst-Case Ripple
30         AA = 4'b0001; BB = 4'b1111; CCin = 0;
31         #20;
32
33         $display("Simulation Completed...");
34         $finish;
35     end
36 endmodule

```

Listing 2: SystemVerilog Testbench for Worst-Case Ripple